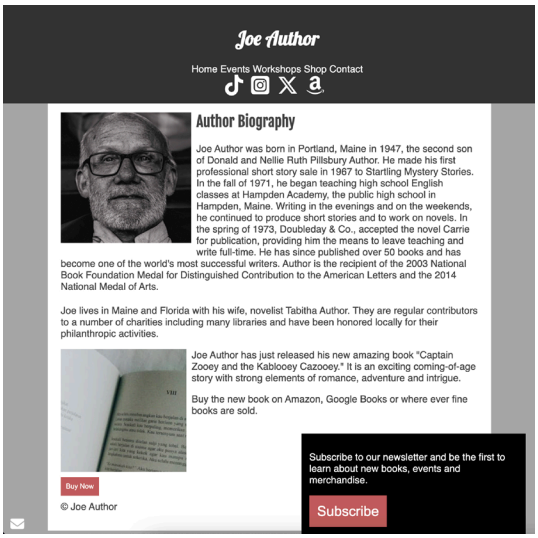
CSS Floats / Position and Icon Fonts.

In the two previous labs you have learned to layout webpages using CSS grids and CSS flexbox. In this lab we will cover additional layout methods including using CSS floats and CSS position. We will also discuss web methods for adding additional fonts and icons.

In this lab you will add layout for a webpage for an author.

Overview



Take a moment to open the *author.html* file and *style.css* file into your coding program. If you look at the HTMLof the webpage you'll see a basic page structure with a *<header>*, *<main>* and *<footer>.* In addition all of the HTML is contained inside a master div with a class of *.container* on it. In this lab you will not be changing this HTML.

```
<body>
    <div class="container">
        <header>…
        </header>
        <main>…
        </main>
        <footer>
            © Joe Author
        </footer>
        <section id="subscribe">…
        </section>
    </div>
</body>

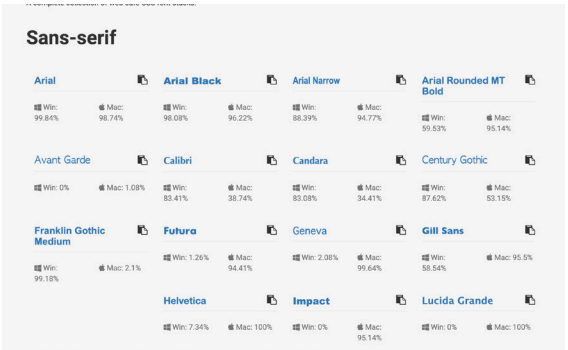
</html>
```

Take a moment to look at the CSS and you'll see that the fonts and backgrounds have been set. You'll also see that the *.container* class is set to width of 800px. You can look at a wireframe to see the general layout of everything.

In this lab we do not need to edit the existing CSS, although we will add to it.

Add a Web Font

One problem that has plagued web developers since the invention of webpages was having a wider range of font choices. Traditonal CSS required that the user have a copy of the font installed on their computer, laptop or smart device. Since most people did not have many fonts installed that left designers a limited range of fonts that they could choose from when writing CSS.



Below is the unofficial list of ***web safe fonts*** or common fonts most users have installed on their device. This is the list many developers used when deciding on font choices.

https://www.cssfontstack.com/

We now have new techniques for adding fonts to our designs and it is called a "web font". A web font is a font installed by the website for the user, it does not require the user to install it, it will be installed automatically. The font is only stored temporarily in the cache of the browser and will be deleted when the browser is closed.

There are many services that add web fonts to a web page, but we will use one called *Google Fonts*. Google fonts provides free use of over 900 font choices.

Google Fonts

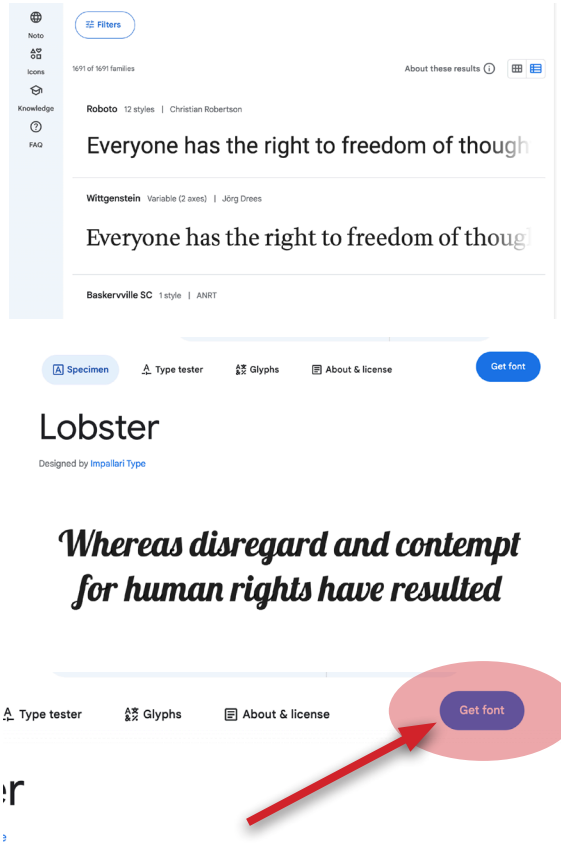
Begin by going to:

https://fonts.google.com/

Please note that Google Fonts interface changes from time-to-time so the photos may not be 100% accurate.

Google fonts allows you to type in custom text and filter fonts by decorative style and classifcation

For this lab we will use the search feature found at the top of the wepbage. Search for a font called "*Lobster*." Your page should look roughly like the image on the right. Click on the name and it give you more information about the font. The "*Speciman"* tab will show you an example of the typeface and show you typeface options. To view the characters or glyphs of the *"glyphs"* tab. To customize the text click on the *"Type Tester*" tab



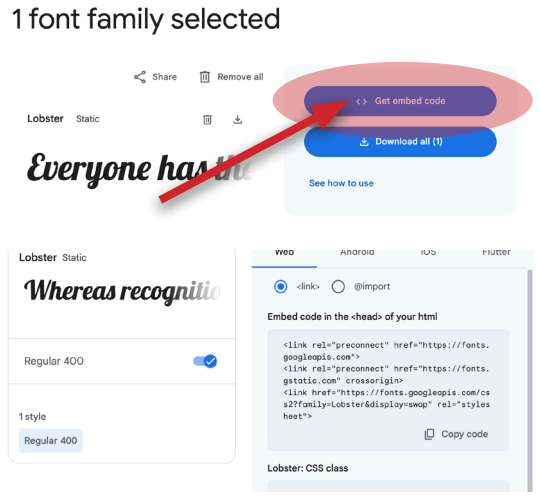
Click the blue button on the upper right that says '*Get Font*'. This adds this font to your collection.

Go to the search feature at the top of the website and search for our second font called *'Fjalla One.'* Use the *"Get Font"* button to add it to your collection.

Add the Google Font to the Project

Now that we have our selected fonts let's use them in our project. Click on the shopping bag icon in the upper right.

Click the option labelled *'Get embed code'* and choose the option marked '*<link>*'. This will give you the HTML code you need to embed the Google fonts into your project. Copy the code and paste it into the *<head>* section of the *author.html* file. You should paste the code **before** the *<link>* tag to *style.css*. The idea is that you should embed the font code before calling it in your CSS




```
<meta charset="UTF-8">

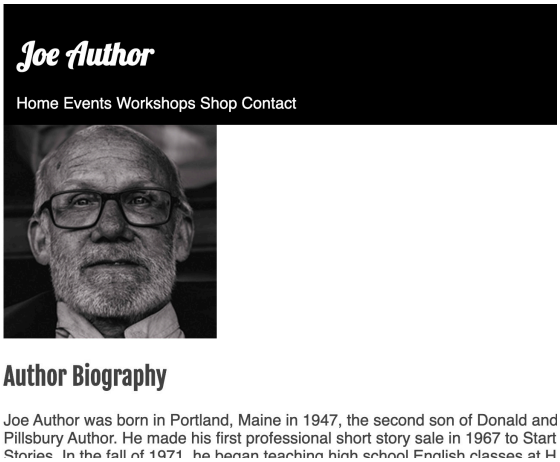
<link rel="preconnect" href="https://fonts.
googleapis.com">
<link rel="preconnect" href="https://fonts.
gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?
family=Fjalla+One&family=Lobster&display=swap"
rel="stylesheet">

<link rel="stylesheet" href="style.css">
```

Use the Font

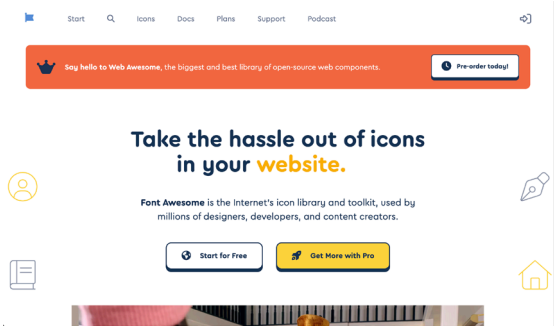
Now that we have installed the Google font, let's call it in our CSS. Go back to the Google font website, under the selection *<link>*, there is a section called '*Lobster: CSS Class'* and *Fjalla One: CSS Class*. It will provide you the CSS you need to call the fonts correctly.

In the CSS set the h1 to *'Lobster'* and set the h2 to *'Fjalla One'*. Save and preview, your page should look roughly like the image on the right.



Use an Icon Font to Build a Social Media Bar

As a developer you'll find that you need icons a lot in your visual designs, icons for websites like brand logos, navigations icons like arrows, or other iconography. Rather that having to make a lot of individual images we will use a web developer technique known as an "*icon font*." An icon font is a font that has icons or images rather than traditional 'a,b,c' characters.



For this lab we will use a popular icon font called *Font Awesome*. To learn more about the font itself you can go to:

https://fontawesome.com/

Please note that the Font Awesome user interface changes from time-to-time so the images may not match 100%

For this lab we are going to add a social media bar for the author that goes to his Tik Tok profile, Twitter / X profile, Instagram profile and Amazon. We can get these icons from Font Awesome

For this lab we are going to use something called a *content delivery network*, or *CDN*. A CDN is a process where rather than storing the files in YOUR root folder you link to someone else's root folder that is stored online. This process is nice because than you have less files to deal with.

The CDN for font awesome should look something like the code below. Instead of copying this from a PDF like these instructions open the document code and copy and paste the code into your *<head>* section.

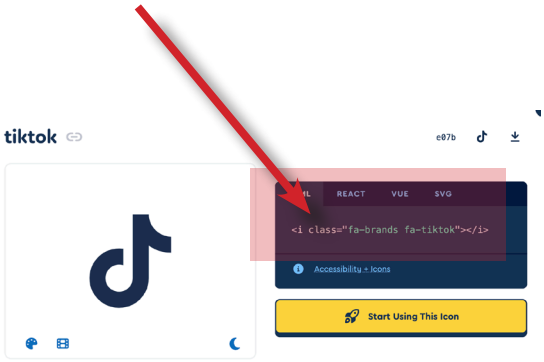
```
<link rel="stylesheet" href="https://cdnjs.
cloudflare.com/ajax/libs/font-awesome/6.5.2/css/
all.min.css">
```

Great! That installs Font Awesome into our project. Now we need to know the HTML to produce specific icons. Font Awesome has

documentation to show you the HTML code you would need to produce an icon. You can find the official gallery below:

https://fontawesome.com/icons?d=gallery

Go to gallery and in the search bar type *'Tik Tok'*. From the results list click on the one that says "Tik Tok", this will open a new dialogue window that shows the icon.



To the right are the code embedding of this icon. Click on the *HTML* tab and copy the entire code.

Inside the *author.html* find the *<nav>* tag. Create a second *<nav>* for the social media icons we are going to add. Then paste in the Font Awesome code.

```
<nav>
     <i class="fa-brands fa-tiktok"></i>
</nav>
```



Wrap the * * inside an *<a>* tag to make it a working hyperlink. Have it when someone clicks the icon go to

https://www.tiktok.com

```
<nav>
     <a href="https://www.tiktok.com"><i
class="fa-brands fa-tiktok"></i></a>
</nav>
```



Find the icons for Instagram, Twitter / X and Amazon. Make them links to their respective website.

Apply CSS

Now that we've added the icons we need to style them. Notice that every Font Awesome ** tag has a class called *.fa-brands* applied to them. We can use that to style all of our Font Awesome icons.

Go to the CSS document and add a class called *.fa-brands*, this will affect all of our icons. Since Font Awesome is a font technically we can use CSS typography to edit our icons, let's use font-size and color to set the size and color of the icons.

Set the class to a text size of 30pt and a color of white. Save and preview and your image should look roughly like the image on the right.

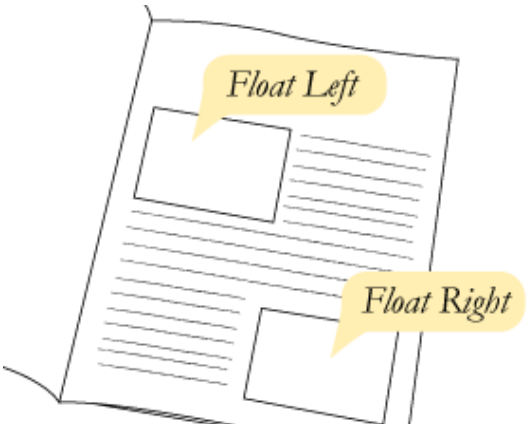


You'll notice they are a bit tight together horizontally speaking. We can fix this issue with applying a bit of margin-right. Add 10px to the .fa-brands. The result should look roughly like the image on the right.



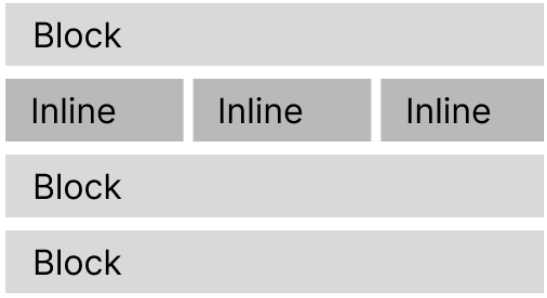
Text Wrap

Take a moment to look at the image of the author. You'll see that the image comes first and then the text about the author is below. It would nice if we could have the text wrap around the image. The reason the text wrap can occur is because of block vs inline element.



Block vs Inline Elements

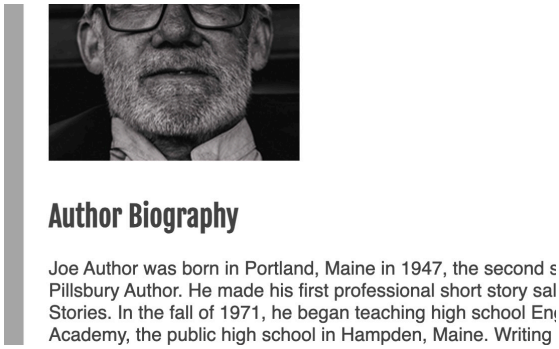
Have you ever wondered why HTML headings like *<h1>*, *<h2>* and *<h3>* stack vertically, but HTML tags like *<a>* like hyperlinks stack horizontally? The answer is because by default HTML tags are either inline elements or block elements. Inline elements like <a>, , and are displayed in the same line, while block elements like *<header>*, *<main>* and *<p>* take up the entire horizontal space, and so they stack vertically.



The reason why text can not wrap around the image is because <p> tags defaulty displayed vertically, so they don't wrap. We are going to change that by using CSS floats.

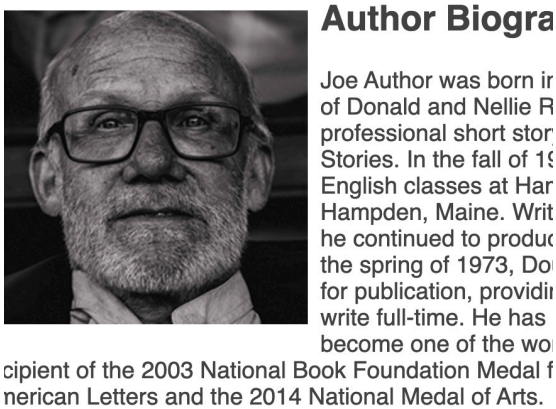
Floats

CSS Floats allow items to be floated to the right or left side of the container they are in. For instance if you have an *<img>* inside a *<section>* the image will be floated to the left or right side of the *<section>*. When you float an HTML item it removes it from the normal layout. The removing allows the HTML below the item to move up in the design. The end result are items that wrap around one another, allowing for more complex layouts.



Create a class called *.floatimage* and tell it to *float: left* as below:

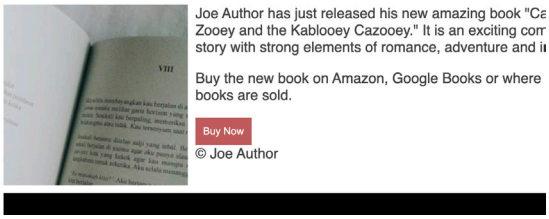
```
.floatimage {
 float: left;
}
```



Apply the CSS class to the image to *author-photo.jpg*. Save and preview and you should notice the text wrapping around the photo of the author.

The text is a bit tight up against the photo. To fix this apply *margin-right* and *margin-bottom* to the *.floatimage*. Add about 10px on the right and bottom side.

Clear



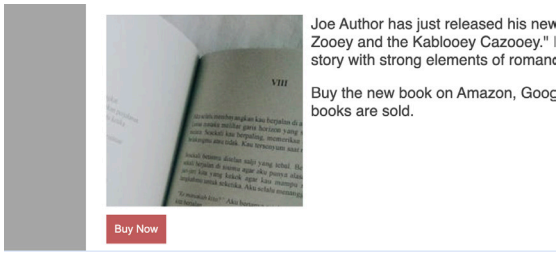
After the author biography there is a section about his new book. There is a photo of his new book and press copy about the book. We would like to wrap the text around the image like we did with the picture of the author. Let's apply the *.floatimage* class we created to the *author-book.jpg*

Save and preview and your webpage should look like the image on the right.

You'll notice that there is a *"Buy Now"* button that is appearing to the right of the image. This button is wrapping around the *author-book* image due to the float being applied. What we would like to do is have the button break free of the wrapping caused by the CSS floats. We can do this by applying what is referred to as a *"clear"*. CSS clears when applied break items that are being wrapped due to CSS floats. Let's create a class to assist us:

```
.clear {
  clear: both;
}
```

When an item recieves the clear it will cause it to move down the webpage to find the next available spot that is not being affected by a CSS float. This spot can the first available spot on the right, left or both, meaning both the left and right need to have available space.



Find the button in the HTML code and apply the *.clear* class. Save and preview and you should see the button moving down below the *author-book* image.

CSS Positioning

When a browser reads and displays HTML it lays it out top → down, left → right, meaning the code at the top of the HTML document is displayed first and the code at the end of the HTML document is displayed last.

Sometimes that is not what we want, we may want HTML to always remain at the top of the browser or at the bottom of the browser regardless of how the. To fix this we can use *CSS positioning*. CSS positioning allows the developer to set where exactly to place an item, regardless of where it is in the code.

In *Preview.pdf* you'll notice the at the authors name and navigation is at the top of the page. In this case, we always want the text to remain at the top, even if people scroll to the bottom of the webpage. To fix this we'll use CSS position.

Begin by creating a CSS rule that affects the *<header>*. Set it to

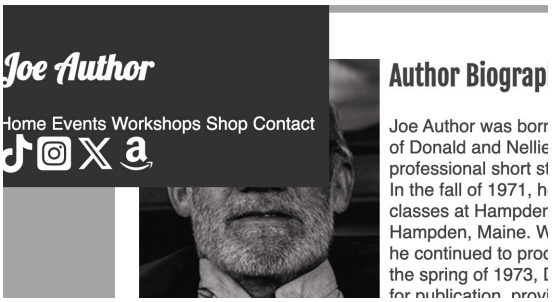
position: fixed.

```
header {
   position: fixed;
}
```



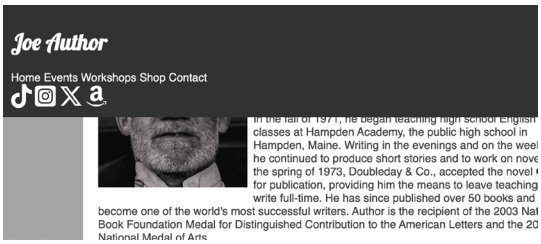
Position fixed means that the item will be placed based on the browser. When can then use top, bottom, left and right to place the item. For instance, ***top: 100px*** would mean place the item 100px from the top of the browser, ***bottom: 100px*** would mean place the item 100px from the bottom of the browser.

We want the *<header>* to stay at the top of the webpage so let's set it top: *0px; left: 0px*. This will make sure the *<header>* stays at the top-left corner of the webpage.



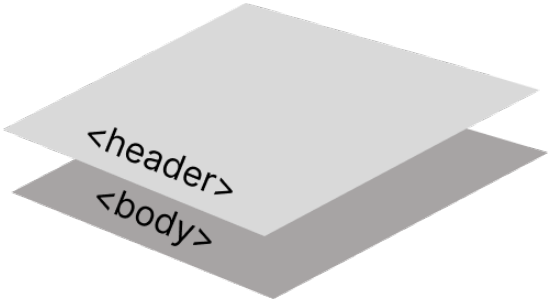

```
header {
   position: fixed;
   top: 0px;
   left: 0px;
}
```

Save and preview. You should see it place it at the upper left. However, its not going across the entire webpage. That's because using CSS position makes HTML elements that are normally block into inline block elements. We can fix this by giving the *<header>* a width. Let's set the width to 100% to make the *<header>* go about 100% of the webpage.




```
header {
   position: fixed;
   top: 0px;
   left: 0px;
   width: 100%;
}
```

Now that we've laid this out, you'll notice that our *<header>* is blocking some of our webpage content. That is because once you start to use CSS positioning you create what is know as ***layers***. Layers in CSS are similar to layers in programs like Adobe Photoshop or Adobe Illustrator in that some content sits in front of other content, so the *<header>* is now blocking my other material.



To fix this we'll push all of the material of the webpage down using padding and body. We can apply padding to the top of the *<body>* add about *150px*.



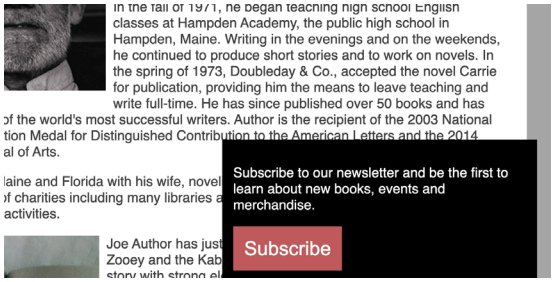
Save and Preview. Your webpage should look roughly like the one on the right.



Use *text-align* for the *<header>* to place the title, navigation items and social-media icons in horizontal center.

Subscribe Call-to-Action

The author has a subscribe call-to-action that he wants always present on the webpage. He wants it in the lower right hand part of the webpage.



Apply *position: fixed* to the *#subscribe* area and use bottom and right to place it accordingly. Give it a *width* of *400px*. Use *Preview.pdf* as a reference.

Submission

Zip the Author folder into a zip file and submit it via Canvas. Please make sure to include all necessary files.

Extra Credit (not required)

The author wants to add an email icon at the lower right hand left hand side of page that when clicked goes to *joeauthor@mail.com*

Use Font Awesome to create an email icon and use *position: fixed* to keep the icon sticky to the bottom left side of the webpage.

